

FRAUD INTELLIGENCE PLATFORM

Behavioral Fraud Detection and Explainable Real-Time Transaction
Analysis

Project Technical Report

Submitted by

Team Member 1: _____ Register No.: _____

Team Member 2: _____ Register No.: _____

Team Member 3: _____ Register No.: _____

Under the guidance of

Guide name and designation: _____

Department: _____

Institution: _____

October 2026

Table of Contents

1. Executive Summary.....	4
2. Problem Statement.....	5
3. Our Understanding of the Problem.....	5
4. Proposed Solution.....	6
5. System Architecture.....	8
6. Technology Stack.....	10
7. Machine Learning Approach.....	10
8. Features Used.....	11
9. Explainable AI: SHAP.....	12
10. Behavioral Analysis.....	13
11. Fraud Ring Detection.....	14
12. Major Dashboard Modules.....	15
12.1 Live Scan.....	15
12.2 Batch Scoring.....	16
12.3 Fraud Rings.....	17
12.4 Model Performance.....	18
13. Data Generation and Dataset.....	18
14. Model Evaluation.....	20
14.1 Why a careful evaluation was necessary.....	20
14.2 Alert budget and metrics.....	20
14.3 Stages of the evaluation.....	20
14.4 Final hold-out results.....	21
14.5 Caveats that must be read with these results.....	22
15. Why Seed-14 Was Not Promoted.....	23
16. Production Model vs Evaluation Candidate.....	24
17. Database and Persistence.....	24
18. API Architecture.....	25
19. Security and Reliability Considerations.....	26
20. Testing.....	27
21. Current Project Results.....	28
22. Limitations.....	28
23. Future Enhancements.....	29
24. Team Contribution.....	31
25. Conclusion.....	32

Terminology Used in This Report

Term	Meaning in this report
Fraud Score	The fraud model's output on a 0 to 100 scale. A model score, not a calibrated probability.
Risk Score	A 0 to 100 temporal behavioral risk signal, produced by the LSTM from the customer's previous ten transactions.
Alert Level	The classification Low, Medium, High or Critical, derived from the Fraud Score.
Behavioral similarity	How closely a transaction matches the customer's own past behavior, as a percentage. Deviation is its complement.
Fraud ring	A cluster of customers connected through shared devices. A potential suspicious relationship, not proof of fraud.
Fraud episode	A group of fraud transactions on one customer that belong to the same incident.
Model set	A named group of model files that the backend loads together.
Production model	The model set loaded by default and used by the running application.
Seed-14 candidate	The model set v2_dnn_lstm_seed14: an evaluation candidate that is not deployed.
Alert budget	The accepted number of alerts on legitimate transactions, set at about 10 per 1,000.
Hold-out data	Data kept apart from training and threshold setting and used only to measure performance.

1. Executive Summary

The Fraud Intelligence Platform is a prototype system that scores banking transactions for fraud risk, explains each score, and helps an investigator see relationships between accounts. It combines two neural networks with an explanation method and an investigation dashboard. It was built as an academic project and is evaluated entirely on synthetic data.

The problem. Fraud is rare, changes over time, and looks different for every customer. A purchase that is routine for one person is alarming for another. Rule-based monitoring handles this poorly, and a system that only answers "fraud" or "not fraud" gives an investigator nothing to verify or act on. Fraud that is coordinated across several accounts is harder still, because no single account looks unusual on its own.

The solution. For each transaction the platform computes nine behavioral features that compare the transaction with that customer's own history. A Long Short-Term Memory (LSTM) network reads the customer's previous ten transactions and produces a **Risk Score**. A dense neural network (DNN) combines the nine features with the Risk Score and produces a **Fraud Score** from 0 to 100, which is mapped to an **Alert Level** (Low, Medium, High or Critical). SHAP attributions show which features raised the score. A separate rule lists devices shared by several customers and presents them as potential **fraud rings**.

Main capabilities.

- **Live Scan:** scoring of a single transaction with Fraud Score, Risk Score, Alert Level, behavioral similarity, SHAP reasons, customer profile and a downloadable PDF report.
- **Batch Scoring:** upload of a CSV file, validation of every row, bulk scoring, filtering by Alert Level and export of the results.
- **Fraud Rings:** a relationship view of customers linked through shared devices.
- **Model Performance:** the evaluation metrics and technical metadata of the model set that is currently loaded.

Technologies. Python, FastAPI, TensorFlow/Keras, SHAP, SQLAlchemy with SQLite, and ReportLab on the backend; React, Vite, Tailwind CSS and Recharts on the frontend.

Current status. The application runs the **production model set**, which is the default. A retrained model, referred to in this report as the **Seed-14 candidate** (v2_dnn_lstm, training seed 14), passed a pre-registered final hold-out evaluation on unseen synthetic data: it alerted on 55.8% of fraud transactions against 27.9% for the production model, with slightly fewer false alerts. It has been integrated so that it can be loaded for controlled testing, but it is **not deployed**. Promotion is blocked by open items that are described in Section 15. The backend test suite contains 564 test cases, all passing.

Scope of the claims in this report

All data in this project is synthetic. The results show how the models behave on generated data with known fraud patterns. They do not establish performance on real banking transactions. The Fraud Score is a model score, not a calibrated probability.

2. Problem Statement

Financial institutions process very large numbers of card and online transactions, of which only a small fraction is fraudulent. Monitoring these transactions raises several connected difficulties.

- **Limits of fixed rules.** Rules such as "flag any payment above a fixed amount" are simple to understand, but they treat all customers alike. They miss fraud that stays under the limit and they flag genuine customers whose normal spending is high.
- **Suspicious behavior is relative.** Whether a transaction is suspicious depends on who makes it: their usual amounts, devices, locations, hours and merchant categories. Detection therefore needs a per-customer baseline.
- **Volume and rarity.** In the project's datasets, fraud is between 0.5% and 0.9% of transactions. Any detector must find these few cases without flooding investigators with alerts on legitimate activity.
- **Need for explanation.** An investigator must be able to justify blocking a payment or contacting a customer. A score without reasons cannot be checked and is difficult to defend.
- **Coordinated fraud.** When one actor controls or attacks several accounts, each account may show only a small anomaly. The signal is in the relationship between accounts, which per-transaction scoring does not see.
- **Two modes of work.** An analyst needs to examine one transaction in depth, and also to screen a whole file of transactions in one pass.

Formal problem statement

Problem statement

Given a stream of customer transactions, design and evaluate a system that (a) assigns each transaction a fraud risk score relative to that customer's own behavioral history, (b) explains the score in terms of identifiable behavioral factors, (c) reveals groups of accounts linked by shared devices, and (d) supports both single-transaction investigation and bulk scoring, with performance measured by a leakage-free evaluation.

This project is a research prototype. It is not a production banking system: it has no real customer data, no user authentication and no integration with payment infrastructure.

3. Our Understanding of the Problem

The team approached fraud detection as a question about behavior, not about individual transaction values. The following ideas shaped the design.

Concept	How we understood it
Transaction-level detection	Every transaction receives its own score at the moment it arrives, so that a decision can be made before the payment completes.
Customer behavioral baseline	Each customer has a normal pattern: typical amount, usual device, home city, active hours and preferred merchant categories. The baseline is built only from that customer's earlier transactions.

Concept	How we understood it
Behavioral deviation	The useful signal is the distance between the current transaction and the baseline, not the raw value. An amount of Rs. 50,000 is ordinary for one customer and extreme for another.
Temporal behavior	Fraud often develops over several transactions, for example small test charges followed by a large withdrawal. The sequence of recent transactions carries information that a single transaction does not.
Device and location changes	A device or location the customer has never used is a common sign of account takeover.
Failed login behavior	Several failed logins shortly before a transaction suggest that someone was guessing credentials.
Transaction velocity	Many transactions within an hour may indicate automated or hurried use of a compromised account.
Merchant category anomalies	A purchase in a category the customer does not normally use is a mild signal that becomes stronger in combination with others.
Explainability	Each alert should state which of these factors contributed and by how much.
Fraud-ring relationships	Accounts that share a device are connected, and connected accounts deserve to be reviewed together.
Investigator workflow	The output is used by a person. It has to present the score, the reasons and the customer context in one place, and allow export for record keeping.

Why "fraud / not fraud" is not enough

A binary label hides everything an investigator needs in order to act. Two transactions with the same label can require different responses: one may be a customer travelling abroad, the other an account takeover. For that reason the platform returns several pieces of information for each transaction.

Output	What it gives the investigator
Fraud Score	How strongly the model flags this transaction, on a 0 to 100 scale, so that cases can be ranked.
Risk Score	A temporal risk signal from the customer's previous 10 transactions, computed before this transaction.
Alert Level	A four-step classification (Low, Medium, High, Critical) that supports triage.
Reasons for the alert	The features that raised the score, from SHAP, so that the alert can be verified against facts.
Customer context	The usual device, home city and history size, and a comparison of this transaction with them.
Relationship information	Whether the customer is linked to other customers through a shared device.

4. Proposed Solution

The solution is a pipeline that takes one transaction as input and produces a scored, explained and stored result. Figure 1 shows the stages; the text form of the same flow follows.

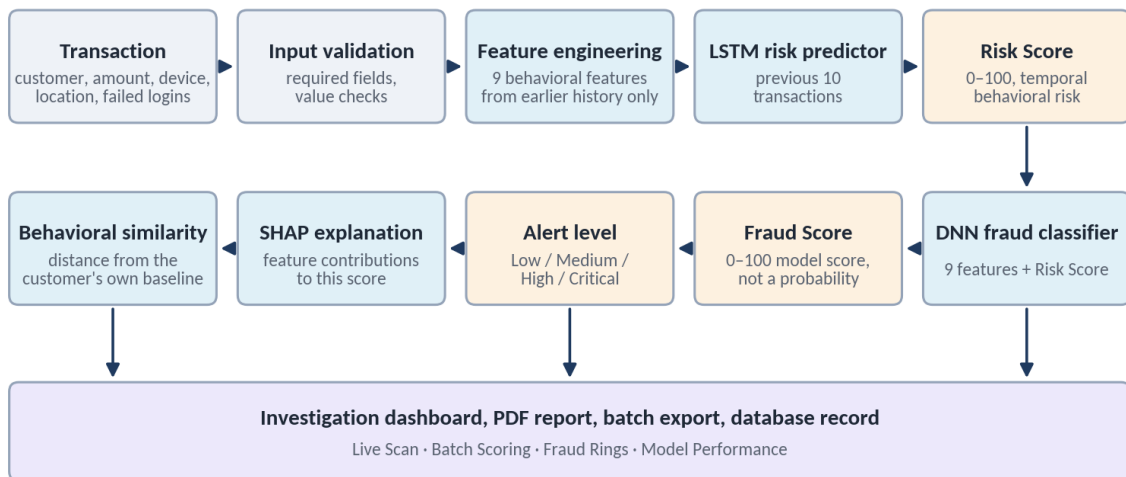


Figure 1. Scoring pipeline for one transaction, from input to investigation outputs.

Transaction

- > Input validation
- > Feature engineering (9 behavioral features)
- > Customer behavioral context (history before this transaction)
- > Temporal risk analysis (LSTM -> Risk Score)
- > Fraud model (DNN)
- > Fraud Score (0-100 model score)
- > Alert classification (Low / Medium / High / Critical)
- > SHAP explanation (reasons)
- > Behavioral comparison (similarity / deviation)
- > Fraud-ring analysis (shared devices, separate path)
- > Investigation dashboard
- > PDF report / batch export

Stage	What happens
Input validation	The request must contain a customer ID, a positive amount, a merchant category, a device ID, a location and a non-negative failed-login count. Invalid requests are rejected with a clear message before any model runs.
Feature engineering	Nine behavioral features are computed for the transaction from the customer's earlier transactions only, so that no future information leaks into the features.
Customer behavioral context	The customer's history is held in time order. It starts from the seed data and includes every transaction scored so far. The ten transactions before the current one form the sequence window.
Temporal risk analysis	The LSTM reads the ten-transaction window and outputs a value that is scaled to the Risk Score (0 to 100). It describes the behavior before the current transaction.
Fraud model	The DNN receives the nine features of the current transaction plus the Risk Score.
Fraud Score	The DNN output is multiplied by 100 and capped at 99.9. It is a model score, not a probability.
Alert classification	Fixed bands on the Fraud Score: below 25 Low, 25 to below 50 Medium, 50 to below 80 High, 80 and above Critical.

Stage	What happens
SHAP explanation	SHAP attributions on the DNN are computed, and the features with the largest positive contributions (up to four) are returned as reasons.
Behavioral comparison	The transaction's feature values are compared with the customer's own historical mean and spread, giving a similarity percentage and its complement, the deviation.
Fraud-ring analysis	Independently of scoring, all customer histories are searched for devices used by two or more customers.
Dashboard, report and export	The result is shown in the Live Scan view, saved to the database with the model set that produced it, and can be downloaded as a PDF. Batch results can be exported as CSV.

5. System Architecture

The platform has a browser frontend and a Python backend that communicate over HTTP with JSON. The backend owns all scoring logic, data and models; the frontend only displays what the backend returns.

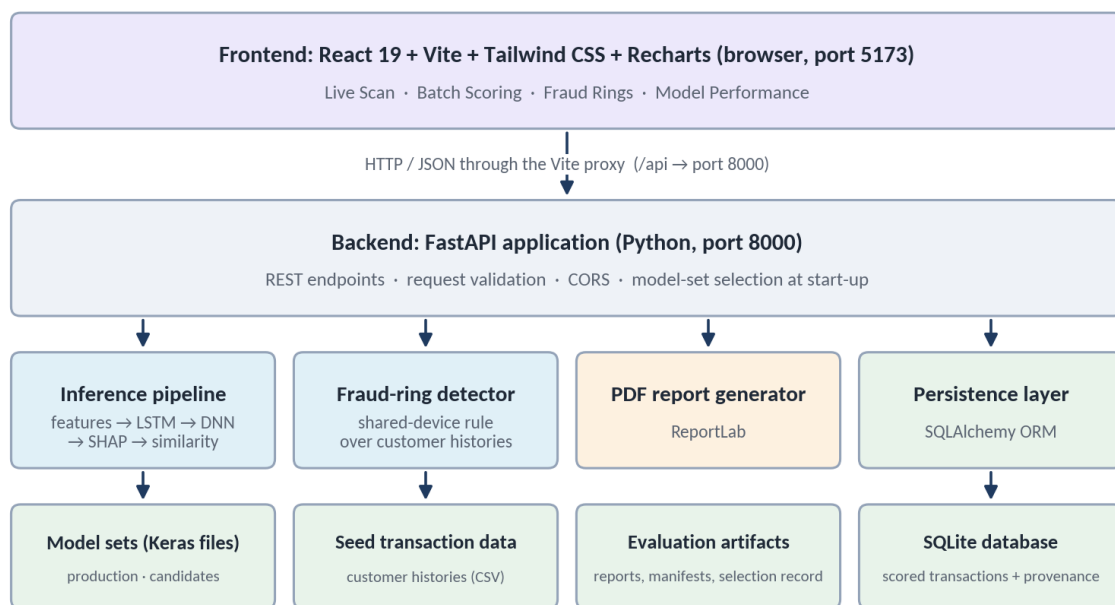
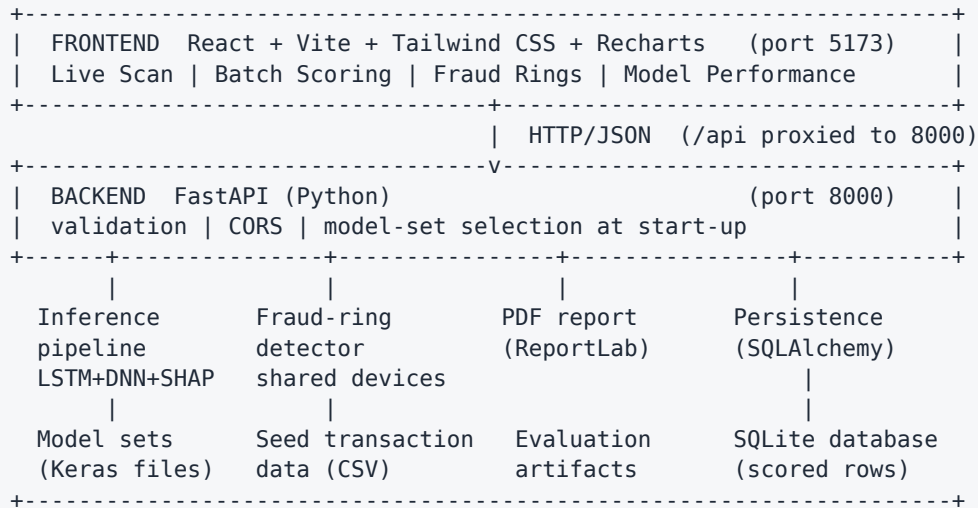


Figure 2. System architecture: frontend, backend services and stored artifacts.



Frontend

- **React** components for each view; **Vite** as development server and build tool; **Tailwind CSS** for styling.
- **Recharts** draws the score timeline. The fraud-ring graphs and the SHAP bars are drawn directly as SVG and HTML.
- All requests go through one API module (`src/api.js`) built on **axios**. In development, Vite forwards `/api/*` to the backend on port 8000.

Backend

- **FastAPI** exposes the REST endpoints listed in Section 18 and validates requests with **Pydantic** models.
- **SQLAlchemy** maps scored transactions to a relational table. **SQLite** is the default database. The database URL is read from the `DATABASE_URL` environment variable and the PostgreSQL driver is listed in the requirements, so PostgreSQL can be configured; the project has only been run and tested with SQLite.
- The model set is chosen once at start-up from the `MODEL_SET` environment variable. If it is not set, the production model set is loaded.

Machine learning

- **TensorFlow / Keras** models: an LSTM temporal risk model and a DNN fraud classifier.
- **SHAP** (GradientExplainer) for feature attributions on the DNN.
- A behavioral similarity computation implemented with NumPy.

Storage

- SQLite database of scored transactions.
- Model artifacts: Keras model files, feature scaling arrays and a SHAP background sample for each model set.
- Evaluation artifacts: JSON reports, dataset manifests and the selection record.

Reporting

- **ReportLab** generates the PDF explanation report for a scored transaction.

6. Technology Stack

Table 1. Technologies used in the project, with versions from the dependency files.

Layer	Technology	Purpose
Frontend	React 19, Vite 8	User interface and development/build tooling
Frontend styling	Tailwind CSS 4	Layout and visual design
Frontend HTTP	axios	Calls to the backend API
Visualization	Recharts 3, inline SVG	Score timeline chart; ring graphs and score rings
Backend	Python, FastAPI 0.139, Uvicorn	REST API and application server
Validation	Pydantic 2	Request and response schemas
Machine learning	TensorFlow 2.21, Keras 3.15	LSTM temporal risk model and DNN fraud classifier
Data processing	pandas, NumPy, scikit-learn	Feature engineering, metrics and data splitting
Explainability	SHAP 0.52	Per-transaction feature attributions
Database	SQLite through SQLAlchemy 2.0	Storage of scored transactions and their provenance
Reporting	ReportLab 5	PDF explanation reports
File upload	python-multipart	CSV upload for batch scoring
Testing	pytest, httpx, pypdf	Backend test suite, API test client and PDF text checks
Code quality	oxlint	Frontend linting

7. Machine Learning Approach

The models form a two-stage design. The first stage summarizes the customer's recent history; the second stage judges the current transaction using that summary.

Stage 1: LSTM temporal risk model

- **Input:** a window of the customer's previous 10 transactions, each described by the 9 behavioral features (shape 10 x 9). The current transaction is not part of the window.
- **Network:** a masking layer, two LSTM layers of 64 and 32 units with dropout, a dense layer of 16 units and a sigmoid output; 31,905 parameters.
- **Training target:** whether the transaction that follows the window is fraudulent.
- **Output:** a value between 0 and 1, multiplied by 100 to give the **Risk Score**.

What we claim about the LSTM

The LSTM processes the transaction sequence to capture temporal behavioral patterns and produces a temporal risk signal for the downstream fraud classifier. We do **not** claim that it predicts fraud before it happens. On v1 it flagged none of the 16 first-fraud transactions in the test period; on v2 the LSTM + DNN design detected the first fraud of an episode less often than a DNN without it (0.605 against 0.690 on the final hold-out).

Stage 2: DNN fraud classifier

- **Input:** 10 values: the 9 features of the current transaction and the Risk Score from stage 1.
- **Network:** dense layers of 64, 32 and 16 units with batch normalization and dropout, and a sigmoid output; 3,585 parameters.
- **Output:** a value between 0 and 1, multiplied by 100 and capped at 99.9, to give the **Fraud Score**.

Training details

- Inputs are standardized with means and standard deviations saved alongside the model. When scoring, standardized DNN inputs are clipped to plus or minus 6.
- Because fraud is rare, the fraud class is weighted more heavily during training. For the v2 models the weight is about 200 to 1.
- For the v2 models, the DNN is trained on out-of-fold LSTM scores: each training row's Risk Score comes from an LSTM that did not see that customer. This prevents the second stage from learning from information the first stage memorized.

Customers with little history

All models were trained only on transactions that have at least 10 earlier transactions. For a customer with fewer, the LSTM is not run and the DNN receives the training-average Risk Score. For a customer's very first transaction, the features that need a baseline are treated as missing. Section 15 explains why this "cold-start" case is an open issue.

Fraud Score is not a calibrated probability

The Fraud Score is the model's output on a 0 to 100 scale. Class-weighted training shifts the outputs upward, so the value is not the frequency of fraud. A Fraud Score of 99.9 means the model ranks the transaction among the most suspicious; it does not mean a 99.9% chance of fraud. The dashboard, the PDF report and the API documentation describe it as a model score for this reason. Calibration is listed as future work.

The **Risk Score** and the **Fraud Score** answer different questions. The Risk Score is a temporal risk signal from the transactions before this one. The Fraud Score describes the transaction itself. A transaction can have a high Fraud Score with a low Risk Score, for example the first fraudulent transaction after a period of normal behavior.

8. Features Used

The nine behavioral features are defined in `backend/app/features/feature_engineering.py`. Each one is computed from the customer's earlier transactions only. The DNN also receives the Risk Score as a tenth input.

Table 2. Model input features.

Feature	What it represents	Why it matters for fraud detection
amount_zscore	How many standard deviations the amount is from the customer's average amount (limited to plus or minus 10).	Fraudulent use often involves amounts far from the customer's norm.

Feature	What it represents	Why it matters for fraud detection
amount_pct_of_avg	The amount as a percentage of the customer's average amount.	Captures the scale of the amount in a way that is comparable across customers.
hour_is_unusual	1 if fewer than 5% of the customer's earlier transactions occurred in this hour of the day.	Activity at hours when the customer is normally inactive is a common takeover sign.
is_new_device	1 if the customer has not used this device in any earlier transaction.	A fraudster normally operates from their own device.
is_new_location	1 if the customer has not transacted from this location before.	An unfamiliar location suggests someone else is using the account.
is_foreign_location	1 if the location is outside the list of ten home cities used by the data generator.	Foreign use combined with other signals is a strong indicator.
category_is_unusual	1 if the merchant category is outside the customer's usual categories.	Fraud frequently targets easily resold goods the customer does not normally buy.
failed_logins_24h	Number of failed login attempts in the preceding 24 hours.	Repeated failures suggest credential guessing before the transaction.
txn_velocity_1h	Number of the customer's transactions in the preceding hour.	Rapid successive transactions are typical of card testing and cash-out.
risk_score	Output of the LSTM for the previous 10 transactions (DNN input only).	Adds a temporal risk signal from the recent transaction sequence to the single-transaction view.

A safeguard in the code (`assert_no_ground_truth`) refuses to start if the fraud label or any of the generator's ground-truth columns appears among the features. This prevents accidental label leakage.

9. Explainable AI: SHAP

SHAP (SHapley Additive exPlanations) is a method that assigns each input feature a contribution to one particular prediction. It is based on Shapley values from cooperative game theory: the contribution of a feature is its average effect on the output when it is added to different combinations of the other features.

How it is used in the platform

- The explainer is applied to the DNN fraud classifier, using a background sample of training rows stored with each model set.
- For every scored transaction, the contributions of the DNN inputs are computed. The features with the largest positive contributions, up to four, are returned as the reasons.
- Inputs that were imputed because the customer had too little history are never reported as reasons.

How the dashboard presents the reasons

The "Why was this transaction flagged?" panel lists each reason with a readable name, the technical feature name, a horizontal bar proportional to the contribution, the contribution value, its share of the listed contributions, and a three-step importance indicator (Primary driver, Strong, Contributing). The

share and the importance indicator are calculated in the interface from the SHAP values returned by the backend.

Example

For a test transaction of Rs. 75,000 in electronics, made from an unknown device in a foreign city after four failed logins, the production model returned the reasons shown in the figure below: New Device (+0.179), Unusual Transaction Time (+0.168), Unusual Merchant Category (+0.126) and New Location (+0.122). An investigator can check each of these against the customer's record. The explainer uses random sampling, so the values differ slightly between runs.

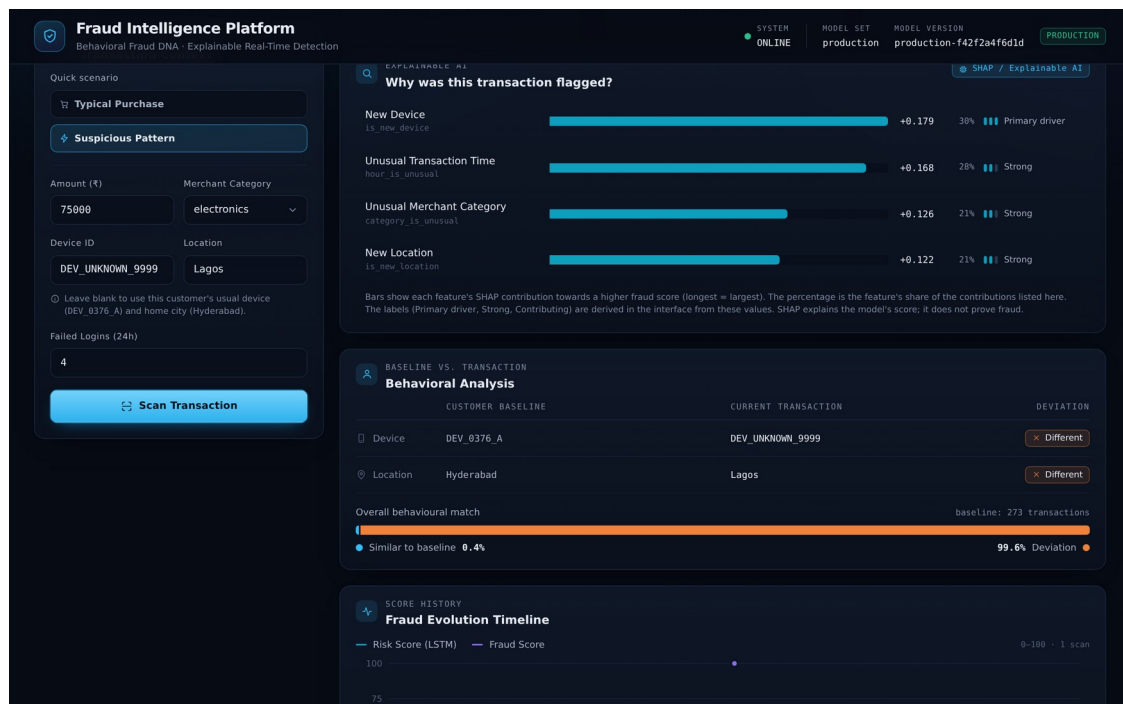


Figure 3. SHAP reasons and behavioral analysis for a scored transaction (production model set, synthetic data).

What SHAP does and does not show

SHAP explains why the model produced its score. It does not independently establish that a transaction is fraudulent. If the model reacts to an irrelevant pattern, SHAP will report that pattern faithfully. The reasons are a starting point for human verification.

10. Behavioral Analysis

Behavioral analysis gives the investigator a direct comparison between the transaction and the customer's normal behavior, alongside the model scores.

Element	How it is obtained
Customer behavioral profile	The usual device is the device most often used in the 90 days before the customer's latest transaction. The home city and the number of transactions in history come from the same history.
Current transaction comparison	The dashboard places the usual device and home city next to the device and location of the scanned transaction and marks each as Match or Different.

Element	How it is obtained
Behavioral similarity	Each of the nine feature values is expressed as a distance from the customer's own mean in standard deviations. The distances are averaged and converted to a percentage with an exponential decay, so that typical behavior is near 100% and very unusual behavior is near 0%.
Deviation	100% minus the similarity.
Risk Score	The LSTM output for the ten transactions before the current one, as described in Section 7.
Score timeline	A chart of the Risk Score and Fraud Score of the customer's previously scanned transactions.

Behavioral similarity is a descriptive indicator. It is not an input to either model. Its value is that it is easy to interpret: it does not depend on trained weights, and it tells the investigator at a glance whether the transaction resembles the customer's past. A pure transaction classifier gives a score; the behavioral view shows what the score is relative to.

11. Fraud Ring Detection

Fraud-ring detection looks at relationships between accounts instead of individual transactions. In the platform it is implemented as a rule over device identifiers, not as a machine-learning model.

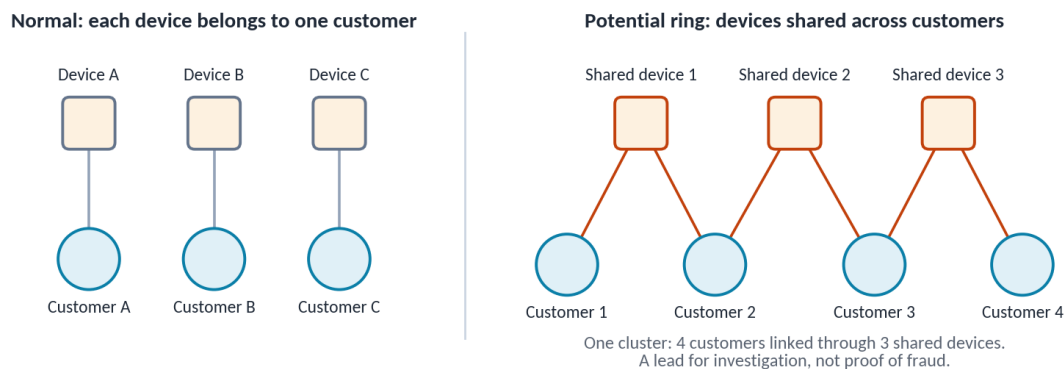


Figure 4. The shared-device idea: independent customers on the left, a connected cluster on the right.

How it works

- **Customer nodes and device nodes.** Every customer and every device is a node. An edge connects a customer to each device that appears in that customer's history.
- **Shared devices.** The backend lists every device used by two or more distinct customers, with the customers involved and the number of transactions on that device (GET /fraud-rings).
- **Clusters.** The interface joins shared devices that have a customer in common into one cluster, so that chains of connected accounts appear as a single group.
- **Investigation summary.** For a selected cluster the interface shows the linked customers, the shared devices, the number of linked transactions and a severity label.

The severity label is derived in the interface from cluster size only (two customers: Medium; three customers or two devices: High; larger: Critical). It is a reading aid and is labelled as such on screen; it is not a model output.

In the seed data used by the application, the rule finds 20 shared devices linking 29 customers, which form 9 clusters. The largest cluster connects 7 customers through 6 devices.

A ring is a lead, not a verdict

A detected ring is a potential suspicious relationship. Family members or colleagues can legitimately share a device. The rule shows which accounts are connected; it does not prove that the connected customers are fraudulent or are victims of fraud.

The rule covers shared devices only. In the final hold-out evaluation, where the true rings are known, the shared-device rule found 18 of the 20 generated rings.

12. Major Dashboard Modules

The dashboard has four tabs. A header shows at all times whether the backend is reachable, which model set is loaded, its version, and whether it is the production model set or an evaluation candidate.

12.1 Live Scan

Live Scan analyses one transaction.

- **Customer selection** and **customer profile**: usual device, home city, transactions in history, scans recorded and the latest scan.
- **Transaction input**: amount, merchant category, device ID, location and failed logins in 24 hours. Leaving the device or location blank uses the customer's usual values. Two quick-fill buttons enter example values.
- **Verdict card**: Alert Level, the rule that produced it, transaction ID, scan time and the PDF download button.
- **Three score cards**: Risk Score, Fraud Score (captioned "Model score, not a calibrated probability") and behavioral similarity with the deviation.
- **SHAP explanations, behavioral analysis** and the **score timeline** below.
- **PDF report**: a document for the scanned transaction with its details, scores, reasons, and the model set and version that scored it.

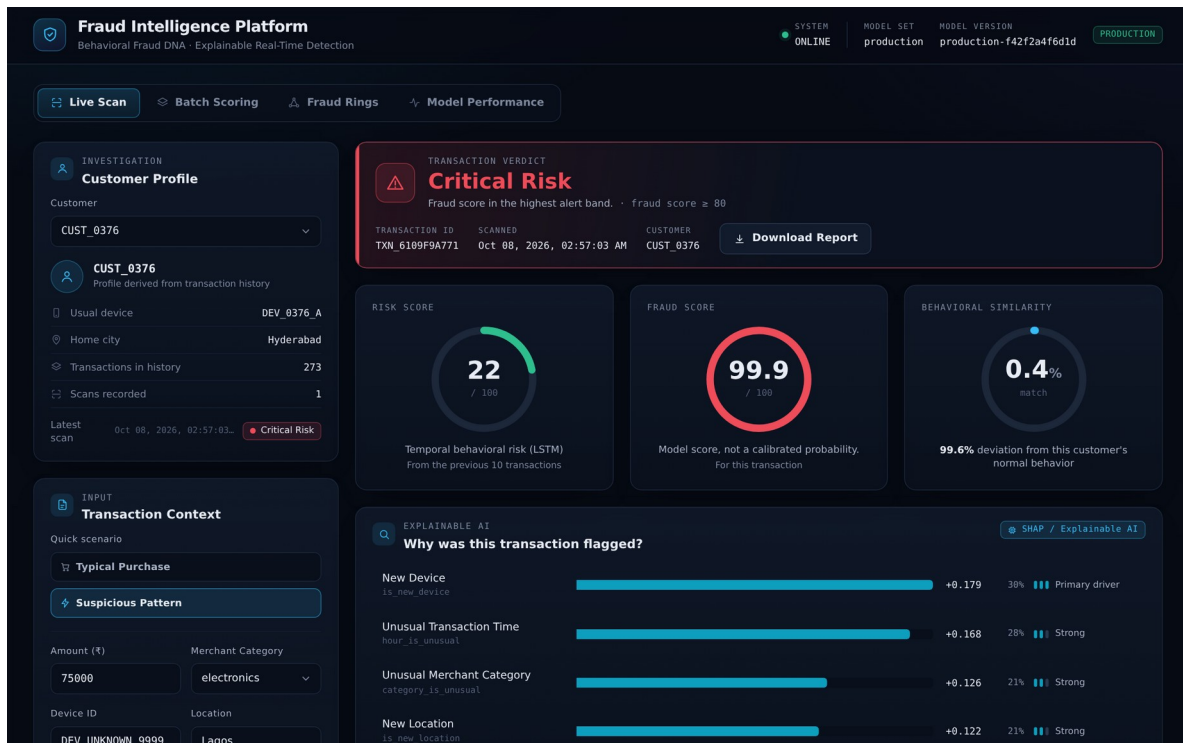


Figure 5. Live Scan: verdict, Risk Score, Fraud Score and behavioral similarity for a test transaction.

12.2 Batch Scoring

Batch Scoring processes a CSV file of transactions in one request. It is intended for screening many transactions, for example a day's export, where opening each one in Live Scan would be impractical.

- **CSV upload** by file picker or drag and drop. Required columns: customer_id, amount, merchant_category. Optional columns: device_id, location, failed_logins_24h.
- **Validation.** The interface checks the header before upload. The backend then validates every row; if any row is invalid, nothing is scored and the response lists the invalid rows with the reason.
- **Bulk scoring** with the same pipeline as Live Scan, one row at a time.
- **Alert distribution:** counts by Alert Level, summary tiles and a stacked bar.
- **Result filtering** by Alert Level and a results table.
- **Export** of the scored results as a CSV file.

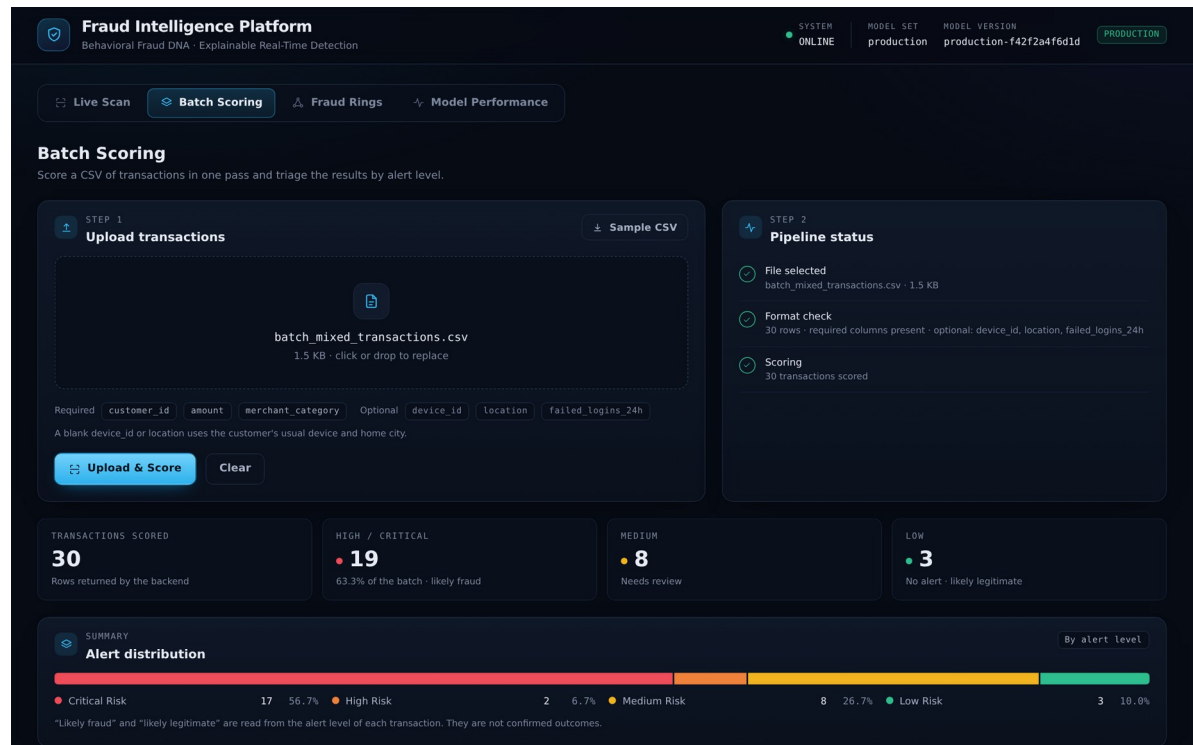


Figure 6. Batch Scoring: upload, pipeline status and summary after scoring a 30-row test file.

12.3 Fraud Rings

- **Ring statistics:** rings detected, affected customers, shared devices and linked transactions.
- **Relationship map:** one graph for each cluster, with customers and shared devices as nodes.
- **Investigation panel:** for the selected cluster, a larger graph, the severity label, the counts and the list of shared devices with the customers on each.

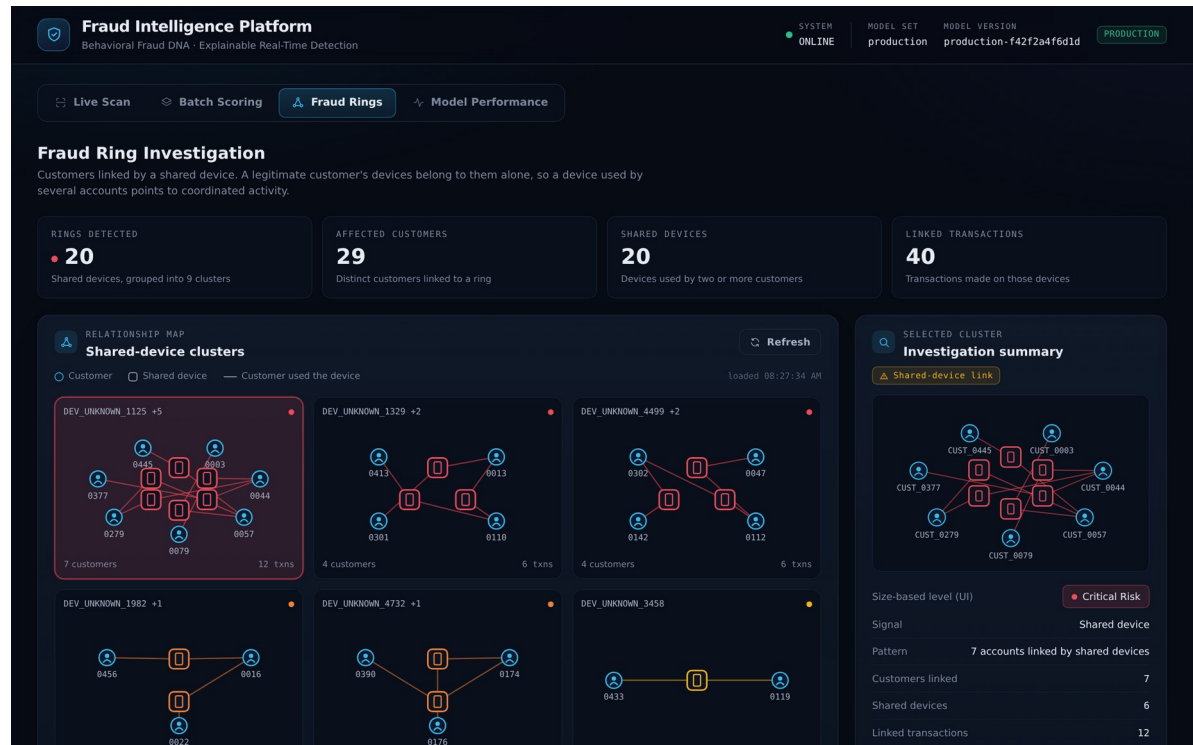


Figure 7. Fraud Rings: statistics, cluster graphs and the investigation summary.

12.4 Model Performance

This tab reports on the model set that the backend has loaded.

- **Production model:** evaluation metrics of the production architecture on dataset v1 (PR-AUC, ROC-AUC, recall, alert rate, fraud caught, confusion matrices) and a model information card with the model set, version, dataset version, feature version, score semantics and alert bands.
- **Evaluation candidate:** when the backend is deliberately started with the Seed-14 candidate, the same tab shows the badges "EVALUATION CANDIDATE" and "NOT DEPLOYED", the final hold-out results for that exact model, the acceptance gates, and the new-customer limitation. In the default configuration no candidate information is shown.

Model performance is kept separate from live scoring on purpose. The metrics describe an evaluation on held-out data; they are evidence about the model. A score shown in Live Scan is one prediction. Mixing the two would invite reading evaluation figures as properties of an individual prediction.

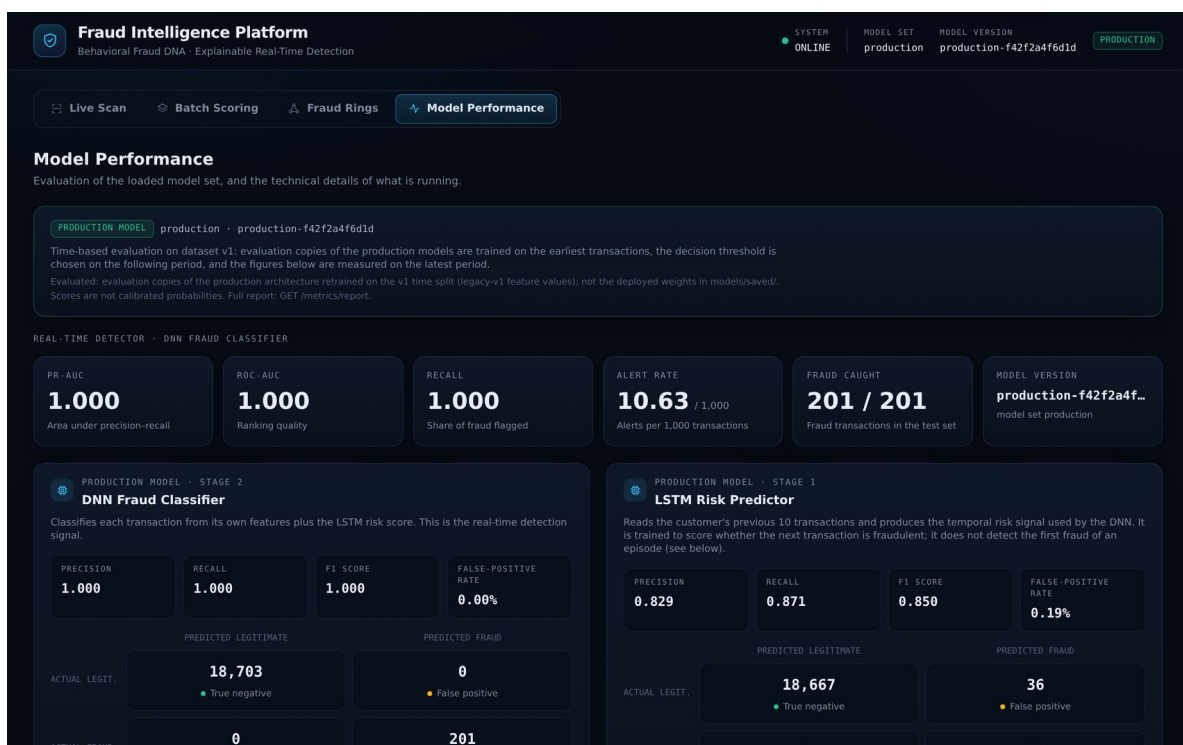


Figure 8. Model Performance for the production model set.

13. Data Generation and Dataset

No real banking data was available to the project, because such data is confidential. All data is synthetic and was generated by scripts in the repository. Three groups of data are used for different purposes.

Table 3. Datasets in the project. Counts are taken from the data files and evaluation reports.

Dataset	Purpose	Customers	Transactions	Fraud	Episodes / rings
v1 (seed data)	History for the running application; training data of the production model	500	93,913	819 transactions (0.87%), 60 customers	60 episodes; 20 shared devices

Dataset	Purpose	Customers	Transactions	Fraud	Episodes / rings
v2 (training and first evaluation)	Training and time-split evaluation of the v2 candidates	500	101,297	471 transactions (0.47%)	110 episodes, 7 fraud types
Hold-out, seeds 101-105	First out-of-sample comparison of candidates	2,500	510,030	2,398 transactions	550 episodes, 20 rings
Development, seeds 301-305	Choosing one trained model (Stage B)	2,500	489,898 evaluated	2,430 transactions	550 episodes
Final hold-out, seeds 401-405	Final acceptance test (Stage C)	2,500	507,292 (482,292 evaluated)	2,446 evaluated	550 episodes, 20 rings
New-customer sets, seeds 411-415	Behavior on customers with little history	2,500	464,312 (25,000 early transactions evaluated)	409 in early transactions	150 new-customer episodes

Why a second dataset was created

On dataset v1 the DNN reached a score of 1.000 on every metric. The team investigated instead of reporting it: a two-condition rule and a logistic regression reached the same 1.000. The cause was the data. Every fraud in v1 was generated from exactly the signals that the features measure, so any reasonable method separates it perfectly. A perfect score there describes the dataset, not the model.

Dataset v2 was designed to be harder and more realistic. Its generator (data/v2/) is deterministic for a given seed and produces:

- Seven fraud patterns: account takeover, stolen card used online, small test charges followed by cash-out, device takeover, a single high-value transaction, fraud that resembles normal activity, and ring fraud.
- Legitimate unusual behavior, such as travel, device upgrades, shared household devices and large purchases, so that unusual does not automatically mean fraudulent.
- Fraud episodes with explicit identifiers, which allow measurement of first-fraud detection and episode detection.
- An optional extension that adds customers who join late and fraud on their first few transactions, used for the new-customer tests.

The hold-out datasets are regenerated from the generator and a seed. They are not stored in the repository; a manifest with checksums is kept for each so that a regenerated dataset can be verified.

Limitation

The current evidence is based on synthetic data and therefore does not establish real-world production performance. The fraud patterns are those the generator was programmed to produce.

14. Model Evaluation

14.1 Why a careful evaluation was necessary

The project's first evaluation used a random 80/20 split of rows. A review found several weaknesses: training rows came from later dates than test rows; all customers appeared in both sets; consecutive LSTM windows overlapped between training and test; the DNN was trained on LSTM scores from a model that had already seen the same rows; and the decision threshold was fixed at 0.5 on uncalibrated outputs. The evaluation was redesigned to remove these problems.

Table 4. Evaluation safeguards.

Concern	Safeguard
Training on the future	Time-based split: train on the earliest period, choose thresholds on the next, test on the latest. A fraud episode is never split across periods.
Same customers in training and test	A second, customer-grouped split; and, for the hold-out, completely new customers generated with different seeds.
Leakage between the two model stages	Out-of-fold stacking: DNN training rows receive Risk Scores from an LSTM that did not see that customer.
Threshold chosen on test data	Thresholds are chosen on the validation period only and then frozen.
One lucky training run	Each architecture was trained with five different random seeds (11 to 15).
Choosing a model after seeing the results	A selection protocol with fixed rules and acceptance gates was written before the selection and final data were generated.
Uncertainty	95% bootstrap confidence intervals, resampling customer groups within each dataset (2,000 resamples).

14.2 Alert budget and metrics

A detector is only usable if investigators can handle its alerts. The evaluation therefore fixes an **alert budget** of about 10 alerts on legitimate transactions per 1,000 transactions (a false-positive rate near 1%). The threshold that meets this budget on validation data is referred to in the project as the Policy B cut-off. A second, stricter cut-off defines a Critical tier. The main metrics are:

- **Recall:** share of fraud transactions that were alerted.
- **Legitimate alerts per 1,000:** alerts raised on legitimate transactions, per 1,000 transactions.
- **Critical recall:** share of fraud transactions that reach the Critical cut-off.
- **First-fraud detection:** share of fraud episodes whose first fraudulent transaction was alerted.
- **Episode detection:** share of fraud episodes with at least one alert.

14.3 Stages of the evaluation

Stage	What was done	Outcome
Production on v1	Time-split evaluation of the production architecture on dataset v1.	DNN: all metrics 1.000 (201 of 201 fraud transactions, 0 false positives). LSTM alone: precision 0.829, recall 0.871, ROC-AUC 0.947. Interpreted as a property of the easy dataset.

Stage	What was done	Outcome
Candidates on v2	Two candidate designs trained on v2: LSTM + DNN (v2_dnn_lstm) and DNN only (v2_dnn_only).	Meaningful, non-perfect scores; basis for all later comparisons.
First hold-out (seeds 101-105)	Seed-42 candidates and production scored on 510,030 new transactions.	Recall: production 0.296, v2_dnn_lstm 0.453, v2_dnn_only 0.420, with fewer legitimate alerts for both candidates. Neither candidate detected more episodes or first frauds than production.
Multi-seed training (seeds 11-15)	Both designs retrained five times.	Mean recall 0.531 (v2_dnn_lstm) against 0.438 (v2_dnn_only); difference +0.092 [0.029, 0.157]. v2_dnn_only was better on first-fraud detection (0.638 against 0.528). Results varied noticeably between training runs.
Stage B: model selection (seeds 301-305)	On separate development data, the training run closest to the median of five metrics was chosen for each design.	Seed 14 chosen for both designs. Cut-offs frozen: 0.7929 (alert) and 0.9431 (Critical) for v2_dnn_lstm seed 14.
Stage C: final hold-out (seeds 401-405)	The two chosen models and production scored once on untouched data against pre-registered gates.	See Table 5 and Figure 9.

14.4 Final hold-out results

The final hold-out contains 482,292 transactions from 2,500 new customers with at least 10 earlier transactions each, including 2,446 fraud transactions in 550 episodes. Each model alerts at its own frozen cut-off.

Table 5. Final hold-out (Stage C). Values in brackets are 95% confidence intervals where the report provides them.

Metric	Production	v2_dnn_lstm seed 14	v2_dnn_only seed 14
Recall	0.279 [0.248, 0.310]	0.558 [0.517, 0.600]	0.419 [0.378, 0.460]
Legitimate alerts per 1,000	10.60 [10.26, 10.95]	9.11 [8.41, 9.83]	10.04 [9.40, 10.74]
Fraud transactions caught (of 2,446)	683	1,365	1,024
Precision	0.118	0.237	0.175
Critical recall	0.155	0.217	0.190
First-fraud detection	0.388	0.605	0.690
Episode detection	0.615	0.715	0.725
PR-AUC	0.155	0.363	0.271
ROC-AUC	0.717	0.938	0.895
Alert cut-off (model output, 0 to 1)	0.9680	0.7929	0.8425

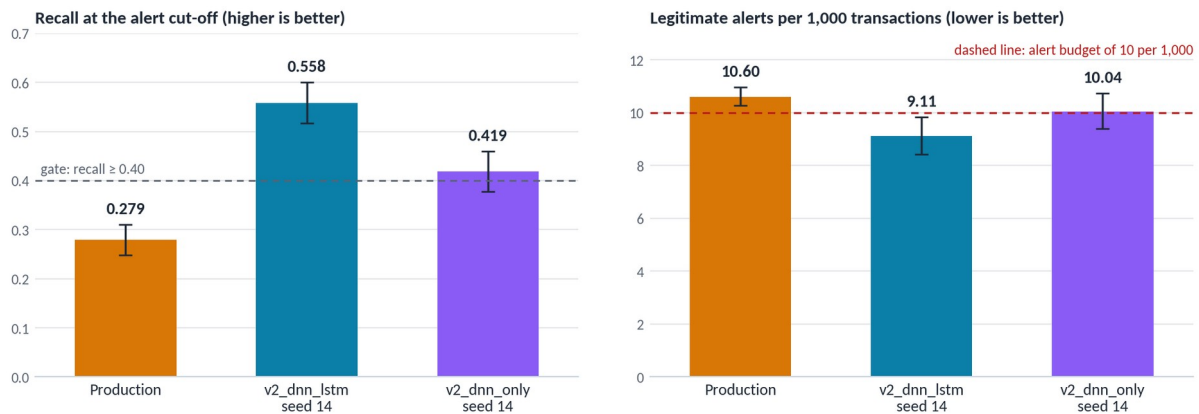


Figure 9. Final hold-out: recall and legitimate alerts per 1,000 with 95% confidence intervals.

Table 6. Pre-registered acceptance gates.

Gate	v2_dnn_lstm seed 14	v2_dnn_only seed 14
Upper bound of legitimate alerts at most 10 per 1,000	9.83: pass	10.74: fail
Recall at least 0.40	0.558: pass	0.419: pass
Lower bound of recall above production's recall (0.279)	0.517: pass	0.378: pass
Result	All gates passed	Not eligible

14.5 Caveats that must be read with these results

- **The alert margin is thin.** The upper bound for the Seed-14 candidate is 9.83 against a limit of 10. In five further datasets it exceeded 10 in one (10.49).
- **First-fraud detection favors the simpler model.** v2_dnn_only seed 14 detects the first fraud of an episode more often (0.690 against 0.605). The LSTM needs earlier suspicious activity to react to.
- **Not every fraud type improved.** On device takeover the Seed-14 candidate detected fewer episodes than production (22 against 32 of 55). Fraud that resembles normal activity is almost undetected by every model (recall 0.01 to 0.10).
- **It is one trained model.** Training runs of the same design differ; retraining would not necessarily reproduce this result.
- **New customers.** On transactions from customers with fewer than 10 earlier transactions, no model keeps the alert budget: 75.6 (production), 25.6 (Seed-14 candidate) and 50.2 (v2_dnn_only seed 14) legitimate alerts per 1,000.
- **Different cut-offs from the live application.** These results use the frozen evaluation cut-offs. The running application classifies with fixed bands (25, 50, 80) for every model set.
- **Synthetic data.** All of the above is measured on generated data.

15. Why Seed-14 Was Not Promoted

Current configuration

MODEL_SET = production. The Seed-14 candidate (v2_dnn_lstm, training seed 14) is an evaluation candidate. It is not the deployed model and it is not the model used in demonstrations.

Why it is eligible

The Seed-14 candidate was selected by a rule fixed in advance, and on data never used for training, threshold setting or selection it passed all three acceptance gates. Its recall was consistent across three independent sets of data (0.573, 0.556 and 0.558). The final report therefore states that it is "eligible for a controlled-promotion decision".

Why it was still not promoted

Passing an evaluation makes a model eligible; it does not make it ready. The project documents these open items, each of which blocks promotion:

Open item	Explanation
New-customer behavior	On customers with fewer than 10 earlier transactions the candidate raises about 25.6 legitimate alerts per 1,000, roughly two and a half times the budget. No new-customer policy has been approved, so the headline performance cannot be claimed for these customers.
Alert-budget margin	The budget is met with a small margin (upper bound 9.83 against 10) and was exceeded in one of five additional datasets. It is not guaranteed on other data.
Threshold policy	The candidate's frozen cut-offs are recorded with the model but are not applied by live scoring, which uses the fixed bands. Applying them is a change to alerting that needs its own approval and tests.
Synthetic-only evidence	Every result comes from generated data. Behavior on real transactions is unknown.
Live data version	The candidate was trained and evaluated on v2 data. The running application holds v1 customer histories. The same cut-offs behave differently on the two populations, and nothing measures the candidate on v1. The data version the application will serve has to be chosen first.
Controlled promotion and rollback	A promotion would have to deploy these exact files, with rollback triggers agreed in advance. The decision belongs to the project owner and has not been taken.

What was done instead

The candidate was integrated for controlled testing. A model-set name, v2_dnn_lstm_seed14, loads it only when explicitly requested, and only if every file matches the recorded SHA-256 checksum. In that mode the interface shows "EVALUATION / NOT DEPLOYED". Returning to production requires only that the MODEL_SET variable is unset. A test confirms that the production model gives identical scores before and after such a switch and that the production model files are unchanged.

16. Production Model vs Evaluation Candidate

Table 7. Comparison of the two model sets, from the model metadata and manifests.

Aspect	Production	Seed-14 candidate
Model-set name	production	v2_dnn_lstm_seed14
Status	Default; loaded when MODEL_SET is not set	Evaluation only; not deployed
Architecture	LSTM temporal risk model + DNN classifier	The same architecture (LSTM + DNN)
Training dataset	Synthetic dataset v1	Synthetic dataset v2 (generator 2.0.1)
Training method	Original recipe: random 80/20 split, in-sample stacking	Time-based split, out-of-fold stacking
Training seed	Not recorded in the model metadata	14
Model version identifier	production-f42f2a4f6d1d	v2_dnn_lstm_seed14-32fc53979e06
Model files	backend/models/saved/	backend/models/candidates_multiseed/v2/seed_14/dnn_lstm/
Integrity check at load	Shape checks of models and scaling arrays	SHA-256 of every file and both weight hashes must match the selection record
Threshold status	Fixed alert bands (25 / 50 / 80) applied by live scoring	Frozen evaluation cut-offs 0.7929 and 0.9431 recorded; not applied by live scoring, which uses the same fixed bands
Final hold-out recall	0.279	0.558
Final hold-out legitimate alerts per 1,000	10.60	9.11
Deployment	Running in the application	None
Purpose	Live scoring and demonstration	Controlled testing and evidence for a future promotion decision

Two further evaluation model sets exist, v2_dnn_lstm and v2_dnn_only (both training seed 42). They are earlier candidates kept for comparison and are also not deployed.

17. Database and Persistence

The backend stores every scored transaction in one relational table, transactions, defined with SQLAlchemy.

Table 8. Structure of the transactions table.

Group	Columns
Identity	transaction_id (primary key), customer_id, timestamp
Transaction input	amount, merchant_category, device_id, location, failed_logins_24h

Group	Columns
Scored output	risk_score, fraud_probability (the Fraud Score; the column name is historical), similarity_pct, deviation_pct, alert_level
Ground truth	is_fraud_actual (optional, unknown for live scans)
Provenance	model_set, model_version

- **Customer history.** At start-up the backend loads the seed transactions into memory as each customer's history. Every scored transaction is appended to the customer's history, so later scores take it into account.
- **Scan persistence.** Scored transactions are written to the database. On restart they are replayed into the in-memory histories, so the system resumes with the same state.
- **Provenance.** Each row records which model set and model version scored it. The PDF report takes the model name from this record, not from whichever model is loaded when the report is requested. Rows scored before provenance was introduced keep empty values and are never back-filled.
- **Schema migration.** A small start-up routine adds the provenance columns to an older database file if they are missing.
- **Concurrency.** Scoring operations on the shared in-memory state are serialized with a lock.
- **Database engine.** SQLite is the default and the only engine the project has been tested with. The connection string is configurable through DATABASE_URL; PostgreSQL is supported by the libraries in use but has not been exercised.

Because scans persist, repeated test scans change the affected customers' later scores. A separate database file can be selected through DATABASE_URL for testing or for evaluating a candidate, which keeps the main database untouched.

18. API Architecture

The backend exposes the following REST endpoints (backend/app/main.py). FastAPI also generates interactive documentation for them.

Table 9. Implemented API endpoints.

Method	Endpoint	Purpose
GET	/health	Liveness check; returns {"status": "ok"}
GET	/customers	List of known customer IDs
GET	/customer/{customer_id}/profile	Usual device, home city and number of transactions in history
GET	/customer/{customer_id}/history	Previously scored transactions of the customer, for the timeline
POST	/predict	Score one transaction; returns scores, Alert Level, similarity and SHAP reasons
POST	/predict/batch	Score an uploaded CSV file; returns a summary and per-row results
GET	/fraud-rings	Devices shared by two or more customers

Method	Endpoint	Purpose
POST	/report/pdf	PDF explanation report for one prediction
GET	/model-info	Metadata of the loaded model set: name, version, status, dataset, features, score semantics, thresholds
GET	/metrics	Evaluation metrics for the loaded model set
GET	/metrics/report	The complete evaluation report for the loaded model set

The response field for the Fraud Score is named `fraud_probability` for backward compatibility. Its documented meaning is a model score, and every user-facing label reads "Fraud Score".

19. Security and Reliability Considerations

The following measures are implemented.

Measure	Implementation
Input validation	Pydantic schemas reject malformed single-transaction requests. Batch files are read as raw text and every cell is checked; an invalid file is rejected as a whole and nothing is scored.
Model validation	When a candidate model set is loaded, its feature list, feature order, input shapes, sequence length and clipping settings are compared with what the application expects.
Checksum verification	Candidate files are compared with the SHA-256 values in their manifest. The Seed-14 candidate must additionally match the checksums recorded at selection time for every file.
Invalid MODEL_SET handling	An unknown model-set name stops application start-up with an error that lists the valid names.
No silent fallback	A requested candidate that is missing or fails validation never results in the production model being loaded in its place.
Label-leakage guard	Feature engineering refuses to run if a ground-truth column is among the features.
Persistence and recovery	Scored transactions are stored and replayed after a restart.
Error handling	API errors return explicit status codes and messages; the interface shows loading, empty and error states, and an offline indicator when the backend cannot be reached.
CORS	Cross-origin access is limited to configured origins; by default only the local development and preview servers.
Separation of frontend and backend	The browser never has access to model files or the database; it receives results only through the API.
Read-only model loading	Loading a model set only reads files; tests confirm that production model files are unchanged after switching model sets.

Not implemented

The prototype has no user authentication or authorization, no encryption of data at rest, no HTTPS configuration, no rate limiting and no audit log of user actions. It is intended to run locally. These are listed as future enhancements.

20. Testing

The backend has an automated test suite written with pytest. It comprises 32 test files with 400 test functions, which expand through parametrization to **564 test cases**. In the most recent full run all 564 passed, with none failed and none skipped.

Table 10. Areas covered by the backend test suite.

Area	What is tested
API endpoints	Health, customers, profile, history, prediction, metrics and PDF report responses.
Batch scoring	Required and optional columns, per-row validation messages, rejection of invalid files, defaults for blank device and location.
Scoring correctness	Score units, alert bands, back-dated transactions, history ordering, home-device calculation, cold-start handling.
Persistence	Storage of scored rows, restoration after restart, provenance columns, schema migration, concurrent requests.
Fraud rings	Detection of shared devices and the structure of the response.
Model sets	Default selection, invalid names, missing files, tampered files and manifests, checksum mismatches, no fallback to production.
Seed-14 candidate	Pinned checksums, metadata and status, scoring with SHAP and similarity, PDF content, and the production to candidate to production switch.
Evaluation code	Data splits, leakage checks, metrics, hold-out harness, multi-seed training, selection protocol and final hold-out logic.
Synthetic data generator	Determinism, structure and sanity checks of the v2 generator and its new-customer extension.
CORS configuration	Parsing and application of allowed origins.

Frontend and end-to-end checks

- **Lint and build.** The frontend passes `oxlint` with no errors or warnings and builds successfully with Vite. The frontend is written in JavaScript; there is no type-checking step and no frontend unit-test suite.
- **End-to-end checks.** Browser-driven checks were run against the running application for all four tabs: customer loading, scanning, SHAP display, PDF download, batch upload with filtering and export, the fraud-ring views and the model performance view, at 1366 x 768 and 1920 x 1080. These were run during development and are not part of the automated suite in the repository.
- **Regression testing.** The full backend suite was re-run after each change to model-set handling, and a rollback test compares production scores before and after loading the candidate.

One defect found by end-to-end use illustrates the value of these checks: batch files that took longer than 15 seconds to score were reported as failed by the interface although the backend completed them. The cause was a request time limit in the frontend, which was raised for the batch request.

21. Current Project Results

What the system demonstrates

- An end-to-end working prototype: a transaction entered in the browser is validated, scored by two neural networks, explained with SHAP, compared with the customer's baseline, stored with its provenance and reported as a PDF.
- A model-management mechanism in which the default model is protected and alternative models can be loaded only explicitly and only after integrity checks.

What the model evaluation showed

- On the first dataset, perfect scores turned out to be an artefact of data that was too easy. This was detected, documented and addressed with a harder dataset.
- On unseen synthetic v2 data, the Seed-14 candidate alerted on 1,365 of 2,446 fraud transactions (recall 0.558) with 9.11 legitimate alerts per 1,000; the production model alerted on 683 (recall 0.279) with 10.60.
- The comparison was made under a pre-registered protocol with confidence intervals, and unfavorable findings were reported alongside favorable ones.

What the dashboard demonstrates

- That scores, reasons and customer context can be presented together in a form an investigator can read quickly.
- Consistent wording: the Fraud Score is always described as a model score, and the loaded model set and its status are always visible.

What the fraud-ring module demonstrates

- That a simple relationship rule exposes groups of connected accounts that per-transaction scoring would treat separately: in the seed data, 29 customers in 9 clusters.

What batch scoring demonstrates

- That the same pipeline can screen a file of transactions with strict validation, triage by Alert Level and export. A 30-row test file was validated, scored and exported end to end.

22. Limitations

- **Synthetic data.** All training and evaluation data is generated. The results do not establish real-world performance.
- **Fraud Score is not a calibrated probability.** It supports ranking and banding, not statements about likelihood.
- **New customers.** Customers with fewer than 10 earlier transactions are scored with a fallback, and no model keeps the alert budget for them. A policy and further evidence are needed.
- **Alert-budget sensitivity.** The alert rate varies with the training run and with the data. The Seed-14 candidate met the budget with a small margin.

- **Production model unchanged.** The deployed model is the original one, trained on the easy v1 dataset. On the harder v2 hold-out its recall is 0.279.
- **Evaluation cut-offs are not live.** The running application uses fixed alert bands that were not derived from an evaluation of the loaded model.
- **Sensitivity to time of day.** The unusual-hour feature compares the current hour with the hours in which the customer was active. In testing with the production model, ordinary transactions at an hour unusual for the customer were often classified Medium or higher.
- **Training and live features differ for production.** The production model was trained on feature values from an older version of the feature code; live scoring uses the current version. The effect was audited and is small, but it is a known inconsistency.
- **No pre-fraud prediction claim.** The LSTM is not shown to predict fraud before it happens; the defensible claim is temporal behavioral risk detection.
- **First-fraud detection.** The LSTM-based candidate is weaker than the DNN-only design at detecting the first fraud of an episode.
- **Hard fraud types.** Fraud that resembles normal activity is almost undetected by every model.
- **Fraud rings are rule-based.** Only shared devices are considered, legitimate sharing is not distinguished from fraudulent sharing, and the severity label reflects cluster size only.
- **Simplified geography.** Any location outside ten listed home cities is treated as foreign, including other Indian cities.
- **Prototype infrastructure.** Single machine, SQLite, in-memory customer histories, row-by-row batch scoring (about a third of a second per transaction on the test machine), no authentication.
- **Scans alter history.** Every scan, including test scans, becomes part of the customer's history and affects later scores.

23. Future Enhancements

Planned / Future Enhancement

Nothing in this section is implemented. These are possible next steps, several of which correspond to the open items documented in the project.

Area	Planned / future enhancement
Real-world data	Evaluate on real or public benchmark transaction data; run a candidate in shadow mode beside an existing system before acting on its alerts.
Probability calibration	Calibrate model outputs so that a score can be read as an estimated likelihood.
New-customer policy	Define and evaluate a rule for customers with little history.
Production threshold policy	Approve alert cut-offs derived from evaluation and apply them in live scoring in place of the fixed bands.
Controlled promotion	Resolve the open items in Section 15 and promote a candidate with agreed rollback triggers.
PostgreSQL deployment	Run and test the system on PostgreSQL.

Area	Planned / future enhancement
Authentication and authorization	User accounts, roles and an audit log.
Real-time streaming	Consume transactions from a message stream instead of individual requests.
Advanced graph analytics	Use further link types (addresses, merchants, networks) and graph algorithms for ring detection.
Case and alert management	Assignment, status tracking and investigator feedback on alerts.
Cloud deployment and monitoring	Containerized deployment, performance monitoring and drift detection with a retraining schedule.
Faster batch scoring	Vectorized scoring of batches instead of row-by-row processing.

24. Team Contribution

This section is to be completed by the team. No contributions have been filled in.

Team Member 1: _____ Register No.: _____

Team Member 2: _____ Register No.: _____

Team Member 3: _____ Register No.: _____

Table 11. Suggested contribution table (to be filled in manually).

Work area	Team member(s)	Description of contribution
Problem study and requirements		
Synthetic data generation		
Feature engineering		
Model development (LSTM, DNN)		
Explainability (SHAP) and behavioral analysis		
Backend API and database		
Fraud-ring detection		
Frontend dashboard		
Model evaluation and selection		
Testing		
Documentation and report		

Signatures:

Team Member 1

Team Member 2

Team Member 3

Project Guide: _____ Signature: _____

25. Conclusion

Detecting fraud requires judging each transaction against the behavior of the customer who made it, explaining the judgement, and noticing when several accounts are connected. The Fraud Intelligence Platform addresses these three needs in one working prototype.

Nine behavioral features describe how a transaction differs from the customer's own past. An LSTM models the recent transaction sequence and produces a temporal risk signal, the Risk Score, and a dense network turns the features and that signal into a Fraud Score and an Alert Level. SHAP attributions state which features raised the score, and a similarity measure shows how far the transaction is from the customer's baseline. A shared-device rule presents potential fraud rings for investigation. The dashboard brings these together for a single transaction, and batch scoring applies the same pipeline to a file of transactions with validation, filtering and export.

The evaluation is the part of the project the team considers most important. The first results were perfect, and the team showed that this was because the first dataset was too easy. A harder dataset, a leakage-free methodology, training with several seeds and a pre-registered selection protocol followed. On unseen synthetic data the selected Seed-14 candidate detected about twice as many fraud transactions as the production model at a slightly lower false-alert rate, and it passed all acceptance gates.

The candidate has nevertheless not been promoted. The production model remains the default, because behavior on new customers, the alert-budget margin, the threshold policy and the absence of real data are unresolved. The project therefore ends with a functioning system, a validated candidate that can be loaded for controlled testing, and a documented list of what must be done before it could be deployed. All conclusions apply to synthetic data, and the Fraud Score remains a model score, not a probability.